

Problem: Adaptive Task Scheduler

A computing system maintains a queue of tasks waiting to be executed. Each task contains the following information:

```
Task ID
Start Time
End Time
Priority: HIGH / MEDIUM / LOW
```

For this problem, the required execution time of a task is calculated as:

```
Execution Time = End Time - Start Time
```

For example, if a task has a start time of 3 and an end time of 8, its execution time is 5 units.

The scheduler supports the following three scheduling policies:

- **FCFS (First Come First Served):** Tasks are selected according to their start time. The task with the earliest start time is executed first.
- **Priority Scheduling:** Tasks are selected according to HIGH > MEDIUM > LOW. If multiple tasks have the same priority, the task with the earliest start time is selected first.
- **SJF (Shortest Job First):** The task having the smallest execution time is selected first. If multiple tasks have the same execution time, the task with the earliest start time is selected first.

Preferred Scheduling Policy

When the scheduler is created, the user chooses one of the three scheduling policies as the preferred policy.

```
TaskScheduler scheduler = new TaskScheduler(/* preferred policy */);
```

The preferred policy may also be changed while the program is running.

However, the scheduler is adaptive. Depending on the tasks currently waiting, it may temporarily use a different scheduling policy.

Adaptive Scheduling Rules

Before selecting every next task, the scheduler must examine all tasks currently waiting in the queue and determine which scheduling policy should be used.

Rule 1: Urgent Workload

If at least one waiting task has HIGH priority, the scheduler must use Priority Scheduling, regardless of the user's currently selected preferred policy.

For example, assume the preferred policy is FCFS and the waiting tasks are:

Task	Start	End	Priority
T1	0	8	MEDIUM
T2	1	5	LOW
T3	2	6	HIGH

Since T3 has HIGH priority, the scheduler must temporarily use Priority Scheduling.

Rule 2: Short-Task Workload

If there is no HIGH-priority task waiting, but at least three waiting tasks have execution time less than or equal to 3 units, the scheduler must use SJF.

For example:

Task	Start	End	Priority	Execution Time
T1	1	4	MEDIUM	3
T2	2	4	LOW	2
T3	3	4	MEDIUM	1
T4	4	10	LOW	6

Since at least three tasks have execution time ≤ 3 , the scheduler must temporarily use SJF.

Rule 3: Normal Workload

If neither of the above conditions is satisfied, the scheduler should use the preferred scheduling policy selected by the user.

For example, if the preferred policy is FCFS and there are no HIGH-priority tasks or sufficient short tasks, the scheduler should execute tasks using FCFS.

Important Behaviour

The scheduling decision must be made again after each task is executed.

Therefore, the policy being used may change several times while processing the same queue.
Consider the following situation where the preferred policy is FCFS:

```
FCFS
|
HIGH-priority task appears
|
Priority Scheduling
|
No HIGH-priority task remains
|
Three short tasks are waiting
|
SJF
|
Short-task condition no longer holds
|
FCFS
```

The scheduler should be able to perform these changes without replacing the scheduler object.

Required Operations

Your system should provide suitable operations for at least the following actions:

```
addTask(...)
setPreferredPolicy(...)
executeNextTask()
executeAll()
```

addTask(...) adds a new task to the waiting queue.

setPreferredPolicy(...) changes the scheduling policy preferred by the user.

executeNextTask() should examine the current workload, determine the policy to use, select the appropriate task, execute/remove it, and display the selected task and policy used.

executeAll() repeatedly executes tasks until the waiting queue becomes empty.

Example Scenario

Suppose the preferred policy is FCFS, and the following tasks are added:

Task	Start	End	Priority
T1	0	8	MEDIUM
T2	1	4	LOW
T3	2	4	MEDIUM
T4	3	4	LOW
T5	4	9	HIGH

When the scheduler selects the next task, it must first determine which scheduling policy is currently appropriate.

As tasks are removed from the queue, the workload conditions may change. Therefore, the scheduler may use different scheduling policies for subsequent tasks.